05-01 Bank 및 BankForks 시스템 분석

개요

Solana의 Runtime 시스템에서 Bank와 BankForks는 블록체인 상태 관리의 핵심 컴포넌트입니다. Bank는 단일 슬롯(블록)의 상태를 나타내며, BankForks는 여러 Bank 인스턴스들을 DAG(Directed Acyclic Graph) 구조로 관리하여 포크 처리와 합의를 지원합니다.

Bank 구조체 분석

핵심 구조체 정의

```
#[derive(AbiExample, Debug)]

pub struct Bank {

/// References to accounts, parent and signature status

pub rc: BankRc,

/// A cache of signature statuses

pub status_cache: Arc<RwLock<BankStatusCache>>,

/// FIFO queue of `recent_blockhash` items

blockhash_queue: RwLock<BlockhashQueue>,

/// The set of parents including this bank

pub ancestors: Ancestors,

/// Hash of this Bank's state. Only meaningful after freezing.

hash: RwLock<Hash>,
```

```
/// parent's slot
parent_slot: Slot,
/// Bank slot (i.e. block)
slot: Slot,
/// Bank epoch
epoch: Epoch,
/// Bank block_height
block_height: u64,
/// The pubkey to send transactions fees to.
collector_id: Pubkey,
/// Track cluster signature throughput and adjust fee rate
pub(crate) fee_rate_governor: FeeRateGovernor,
/// cache of vote_account and stake_account state for this fork
stakes_cache: StakesCache,
/// staked nodes on epoch boundaries
epoch_stakes: HashMap<Epoch, EpochStakes>,
/// Protocol-level rewards that were distributed by this bank
pub  rewards: RwLock<Vec<(Pubkey, RewardInfo)>>,
/// Transaction processor for executing transactions
transaction_processor: TransactionBatchProcessor<BankForks>,
// ... 기타 필드들
}
```

Bank 생명주기

Bank는 다음과 같은 생명주기를 가집니다:

1. **생성 (Creation)**: 새로운 Bank 인스턴스 생성
2. **트랜잭션 처리 (Transaction Processing)**: 트랜잭션 실행 및 상태 업데이트
3. **동결 (Freezing)**: 더 이상의 상태 변경 금지
4. **루팅 (Rooting)**: 최종 확정 상태로 전환

Bank 생성 메서드

```
/// Create a new bank that points to an immutable checkpoint of another bank.
pub fn new_from_parent(parent: Arc<Bank>, collector_id: &Pubkey, slot: Slot) -
Self::_new_from_parent(
parent,
collector_id,
slot,
null_tracer(),
NewBankOptions::default(),
)
}
```

새로운 Bank는 항상 부모 Bank로부터 생성되며, 부모의 상태를 상속받습니다.

트랜잭션 처리

```
/// Process a Transaction. This is used for unit tests and simply calls the vect
```

```
/// Bank::process_transactions method.
pub fn process_transaction(&self, tx: &Transaction) -> Result<()> {
self.try_process_transactions(std::iter::once(tx))?[0].clone()?;
tx.signatures
.get(0)
.map_or(Ok(()), |sig| self.get_signature_status(sig).unwrap())
}
```

Bank는 트랜잭션을 처리하여 계정 상태를 업데이트하고 결과를 기록합니다.

Bank 동결 (Freeze)

```
pub fn freeze(&self) {
// This lock prevents any new commits from BankingStage
let mut hash = self.hash.write().unwrap();
if *hash == Hash::default() {
// finish up any deferred changes to account state
self.collect_rent_eagerly();
self.distribute_transaction_fees();
self.distribute_rent_fees();
self.update_slot_history();
self.run_incinerator();

// freeze is a one-way trip, idempotent
```

```
self.freeze_started.store(true, Relaxed);

*hash = self.hash_internal_state();

self.rc.accounts.accounts_db.mark_slot_frozen(self.slot());

}

}
```

동결 과정에서는:

- 지연된 계정 상태 변경사항 완료
- 트랜잭션 수수료 분배
- 렌트 수수료 분배
- 슬롯 히스토리 업데이트
- 소각 프로세스 실행
- Bank 해시 계산 및 저장

BankForks 구조체 분석

핵심 구조체 정의

```
pub struct BankForks {

banks: HashMap<Slot, BankWithScheduler>,

descendants: HashMap<Slot, HashSet<Slot>>,

root: Arc<AtomicSlot>,

pub snapshot_config: Option<SnapshotConfig>,

pub accounts_hash_interval_slots: Slot,
```

```
last_accounts_hash_slot: Slot,
in_vote_only_mode: Arc<AtomicBool>,
highest_slot_at_startup: Slot,
scheduler_pool: Option<InstalledSchedulerPoolArc>,
}
```

포크 관리 메커니즘

BankForks는 다음과 같은 핵심 기능을 제공합니다:

1. 루트 설정 (Set Root)

```
pub fn set_root(
&mut self,
root: Slot,
accounts_background_request_sender: &AbsRequestSender,
highest_super_majority_root: Option<Slot>,
) -> Vec<Arc<Bank>> {
let (removed_banks, set_root_metrics) = self.do_set_root_return_metrics(
root,
accounts_background_request_sender,
highest_super_majority_root,
);
// 메트릭 수집 및 로깅
removed_banks
```

```
}
```

루트 설정 과정에서는:

- 새로운 루트 슬롯 지정
- 루트가 아닌 포크들 제거
- 계정 데이터베이스 정리
- 스냅샷 생성 요청

2. 루트 Bank 접근

```
pub fn root_bank(&self) -> Arc<Bank> {
self[self.root()].clone()
}
```

```
pub fn root(&self) -> Slot {
self.root.load(Ordering::Relaxed)
}
```

3. 프로그램 캐시 정리

```
pub fn prune_program_cache(&self, root: Slot) {
if let Some(root_bank) = self.banks.get(&root) {
root_bank.prune_program_cache(root, root_bank.epoch());
}
```

```
}
```

Bank와 BankForks의 상호작용

포크 생성 과정

1. **새로운 슬롯 도달**: 새로운 슬롯에서 리더가 되거나 다른 검증자의 블록을 받음
2. **Bank 생성**: 부모 Bank로부터 새로운 Bank 인스턴스 생성
3. **BankForks 등록**: 새로운 Bank를 BankForks에 등록
4. **후손 관계 업데이트**: descendants 맵 업데이트

합의 과정

1. **투표 수집**: 각 Bank에 대한 검증자들의 투표 수집
2. **슈퍼 마조리티 확인**: 2/3 이상의 스테이크 투표 확인
3. **루트 설정**: 충분한 투표를 받은 Bank를 루트로 설정
4. **포크 정리**: 루트가 아닌 포크들 제거

성능 최적화

1. 지연 로딩 (Lazy Loading)

Bank는 필요한 시점에만 계정 데이터를 로드하여 메모리 사용량을 최적화합니다.

2. 캐싱 메커니즘

- **StakesCache**: 스테이킹 정보 캐싱
- **StatusCache**: 트랜잭션 상태 캐싱

- **ProgramCache**: 프로그램 실행 환경 캐싱

3. 병렬 처리

트랜잭션 처리 시 병렬 실행을 통해 처리량을 향상시킵니다.

오류 처리 및 복구

1. 상태 일관성 보장

Bank는 트랜잭션 실행 중 오류 발생 시 롤백 메커니즘을 제공합니다.

2. 포크 복구

BankForks는 네트워크 분할이나 합의 실패 시 올바른 포크로 복구하는 메커니즘을 제공합니다.

요약

Bank와 BankForks 시스템은 Solana의 상태 관리와 합의 메커니즘의 핵심입니다:

- **Bank**: 단일 슬롯의 블록체인 상태를 관리하며, 트랜잭션 처리와 상태 업데이트를 담당
- **BankForks**: 여러 Bank 인스턴스를 DAG 구조로 관리하여 포크 처리와 합의를 지원
- **생명주기**: 생성 → 트랜잭션 처리 → 동결 → 루팅의 단계적 진행
- **최적화**: 캐싱, 지연 로딩, 병렬 처리를 통한 성능 향상

이 시스템은 Solana의 고성능과 확장성을 가능하게 하는 핵심 아키텍처 컴포넌트입니다.